AI-Assisted Development Documentation

Campus Cafeteria POS — Computer-Based Food Ordering and Point-of-Sale System

Students	Aaron Jason F. Lanos Joren R. Naungayan
Course	IT 415 – Application Development and Emerging Technologies (Practical Examination)
Scenario	Scenario 1 — Campus Cafeteria POS
Technologies	HTML, CSS, and vanilla JavaScript (index.html, style.css, script.js); in-memory data only
AI tool used	Claude (Anthropic)
Date	October 13, 2026
GitHub Link	https://github.com/Jawren/Campus-Cafeteria-POS

Summary of AI Use

The AI tool was used six times at different stages of development. Each use is documented in the sections that follow with the exact prompt, its purpose, a summary of the AI's response, our evaluation, and any modifications made.

#	Stage	Type of use	Result
1	Initial interface	Building a feature	Correct
2	Cart, totals, change, receipt	Building a feature	Correct
3	Payment validation	Building a feature / edge cases	Correct
4	Review against 12 features	Debugging	Correct (no bugs found)
5	README.md	Documentation	Correct
6	UI polish and refactor	Refactoring	Correct

Prompt 1 — Create the Initial POS Interface

Type of use: Building a feature

Prompt used	The exact prompt text is shown in the screenshots below (Figure 1 and Figure 2).
Purpose of the prompt	Generate the initial desktop interface and project structure: the header, the six products with their exact names and prices, Add to Cart buttons, a cart table with column headings, an order summary showing ₱0.00, a cash input with a Confirm Payment button, and a receipt placeholder. No cart, payment, or receipt logic was to be implemented yet.
AI response (summary)	The AI explained the proposed layout (header, menu on the left, cart, summary and payment on the right, receipt across the bottom) and returned: • complete index.html and style.css • a minimal script.js that only logs a message to confirm it is linked • a short explanation of each file, steps to run the page in a browser, and a manual testing checklist On request, it also packaged the three files into a zip folder.
Our evaluation	**Correct.** All six products and prices matched exactly, every required section appeared, and the constraints were followed (no frameworks, no transaction logic yet). The AI also stated that it had not tested the page in a browser, as the prompt required.
Modifications we made	No code changes. The files were downloaded as a zip and placed in the Campus-Cafeteria-POS folder.

PROMPT 1: CREATE THE INITIAL CAMPUS CAFETERIA POS INTERFACE

1. CONTEXT

I am a beginner IT student completing an individual IT 415 Application Development and Emerging Technologies practical examination. The project is a Computer-Based Food Ordering and Point-of-Sale (POS) System.

My assigned scenario is Scenario 1: Campus Cafeteria POS. The application will be used by student assistants or cashiers at a walk-up campus cafeteria to take lunch orders.

I am using HTML, CSS, and vanilla JavaScript. My project folder is named Campus-Cafeteria-POS and contains three files: index.html, style.css, and script.js.

This examination must be completed within 150 minutes. I need to understand the code and be able to explain how it works during checking.

2. OBJECTIVE

Create the initial desktop-friendly user interface and project structure for the Campus Cafeteria POS application.

At this stage, focus only on the basic interface. Do not implement the complete shopping cart, payment processing, validation, or receipt generation yet. These will be developed and tested in later stages.

3. REQUIREMENTS

A. Display a clear application header with the name "Campus Cafeteria POS".

B. Display these six products with their exact names and prices in Philippine pesos:

- Rice (Plain): ₱15.00
- Fried Chicken (1 pc): ₱65.00
- Pork Adobo: ₱60.00
- Vegetable Side Dish: ₱35.00
- Iced Tea (cup): ₱20.00
- Bottled Water: ₱20.00

C. Include a product section with a visible Add to Cart button for each product. The buttons may be non-functional at this stage.

D. Include a shopping cart section with headings or placeholders for product name, quantity, unit price, and subtotal.

E. Include an order summary that displays the total amount due, initially ₱0.00.

F. Include a payment section with a cash amount input and a Confirm Payment button. These controls are placeholders at this stage.

G. Include a separate receipt section with a placeholder message indicating that no receipt has been generated yet.

H. Organize the layout so the pr[illegible] list, cart, payment area, and receipt are easy to distinguish on a desktop or laptop [illegible]reen.

Figure 1. Prompt 1 as entered (part 1 of 2)

I. Use readable text, clear labels, consistent spacing, and simple professional styling.
4. CONSTRAINTS

- Use only HTML, CSS, and vanilla JavaScript.
- Do not use React, Vue, Bootstrap, external UI libraries, or other frameworks.
- Do not add a database, backend, login system, inventory management, or online payment gateway.
- Do not add products beyond the six specified products.
- Do not change any product names or prices.
- Keep the design simple and practical for a 150-minute examination.
- Do not implement transaction logic yet.
- Do not overwrite unrelated files.
- Keep the code beginner-friendly and easy to explain.
- Do not claim that the interface has been tested unless it has actually been tested.
- EXPECTED OUTPUT
  Provide the following in order:
- A brief explanation of the proposed interface layout.
- The complete code for index.html.
- The complete code for style.css.
- The complete code for script.js, which should contain only the minimal setup needed at this stage.
- A short explanation of the purpose of each file.
- Step-by-step instructions for running the interface locally in a web browser.
- A short manual testing checklist to verify that all six products, prices, and required interface sections appear correctly.

Label each code block with its correct filename. Ensure the HTML references style.css and script.js correctly.
Do not proceed to the shopping cart implementation. Stop after completing the initial interface and explaining how I should test it.

Show less

Figure 2. Prompt 1 as entered (part 2 of 2)

Prompt 2 — Cart, Totals, Change, and Receipt

Type of use: Building a feature

Prompt used	The exact prompt text is shown in the screenshots below (Figure 3 and Figure 4).
Purpose of the prompt	Implement one complete transaction: add to cart (merging duplicates), increase/decrease quantity, remove items, live subtotals and total, cash payment, change computation, payment confirmation, a digital receipt with a TXN-0001 transaction number, and a New Transaction button.
AI response (summary)	The AI returned full updated index.html, style.css, and script.js with: • an in-memory cart array of { id, name, price, qty } • addToCart, increaseQty, decreaseQty, removeFromCart, computeSubtotal, computeTotal, renderCart, processPayment, renderConfirmation, renderReceipt, and newTransaction • helpers such as roundMoney, formatPeso, and formatTransactionNumber • a lock after payment so the same order cannot be paid twice It also gave a short explanation of each function and step-by-step tests. It ran the worked example in a simulated browser: ₱165.00 total, ₱200.00 paid, ₱35.00 change, then TXN-0002 on the next sale.
Our evaluation	**Correct.** The worked example matched exactly (2 Fried Chicken + 1 Rice + 1 Iced Tea = ₱165.00; ₱200.00 paid → ₱35.00 change). As the prompt asked, there was no validation yet, so a blank cash amount showed ₱NaN; this was expected and fixed in Prompt 3. The lock after payment went slightly beyond the prompt, but it prevents a double payment, and Prompt 3 later required it.
Modifications we made	No code changes. The three files were replaced with the updated versions (also requested as separate files for easier copy-paste).

CONTEXT:
I am building a Campus Cafeteria POS web app using plain HTML, CSS, and vanilla JavaScript (index.html, style.css, script.js). Data is kept in memory only. The project already displays 6 hard-coded products with "Add to Cart" buttons, an empty Cart/Order Summary section, and an empty Payment & Receipt section.

OBJECTIVE:
Implement the core ordering and computation functionality so one complete transaction works: add to cart, adjust quantity, remove, live subtotals and total, enter cash payment, compute change, show payment confirmation and a digital receipt, and start a new transaction.

REQUIREMENTS:
1. Cart is an in-memory array of { id, name, price, qty }.
2. "Add to Cart": if the item is already in the cart, increase qty by 1; otherwise add it with qty 1. Show a short success message (e.g. "Fried Chicken (1 pc) added to cart.").
3. Each cart row shows: item name, unit price, a "−" button, qty, a "+" button, subtotal (price × qty), and a "Remove" button.
4. "+" increases qty. "−" decreases qty but never below 1 (disable "−" at qty 1). "Remove" deletes the item from the cart.
5. Whenever the cart changes, automatically re-render the order summary and recompute every subtotal and the TOTAL AMOUNT DUE.
6. When the cart is empty, show "Cart is empty." and a total of ₱0.00.
7. Payment section: a numeric input (type="number", step="0.01", min="0") for "Amount Paid (Cash)" and a "Confirm Payment" button.
8. On Confirm Payment (assume a valid amount for now): change = amountPaid − total.
9. After payment, show a Payment Confirmation box ("Payment Successful") with the total, amount paid, change, and transaction number.
10. Below it, show a Digital Receipt with: store name, transaction number, date and time, a table of items (name, qty, unit price, subtotal), total amount, amount paid, and change.
11. Transaction number format: TXN-0001, incrementing by 1 per completed transaction (in-memory counter).
12. A "New Transaction" button clears the cart, the payment input, the confirmation, the receipt, and all messages, and keeps the transaction counter.
13. Worked example that must match exactly: 2 Fried Chicken + 1 Rice (Plain) + 1 Iced Tea = ₱165.00; paid ₱200.00 → change ₱35.00.

Figure 3. Prompt 2 as entered (part 1 of 2)

CONSTRAINTS:
- Vanilla JS only, no libraries, no localStorage.
- Keep separate, small, named functions (e.g. addToCart, increaseQty, decreaseQty, removeFromCart, renderCart, computeTotal, processPayment, renderReceipt, newTransaction).
- Avoid floating-point errors: round money to 2 decimals (e.g. Math.round(value * 100) / 100).
- Reuse formatPeso for all amounts.
- Use addEventListener, no inline onclick.
- Do not implement payment validation yet (next step). Do not add discounts, tax, login, or inventory.
- Short comments above each function; beginner-readable.

EXPECTED OUTPUT:
1. The complete updated index.html, style.css, and script.js (full files).
2. A brief explanation of each new function.
3. Step-by-step manual test steps using the worked example (₱165.00 total, ₱200.00 paid, ₱35.00 change).

Show less

Figure 4. Prompt 2 as entered (part 2 of 2)

Prompt 3 — Payment Validation and Edge Cases

Type of use: Building a feature / handling edge cases

Prompt used	The exact prompt text is shown in the screenshots below (Figure 5 and Figure 6).
Purpose of the prompt	Add payment validation with clear on-screen messages: reject an empty cart, blank, non-numeric, or negative amounts, and insufficient payment; accept exact and decimal payments; never create a receipt or a new transaction number for an invalid payment; lock the cart after payment; and support the Enter key.
AI response (summary)	The AI returned the full updated script.js, plus small additions to index.html (a payment message area) and style.css (a red error style). It added: • validatePayment(amountInput, total), returning { valid, message, amount } and checking the rules in the required order • red error and green success messages that clear on the next action • if (isPaid) return; guard lines so a paid cart cannot change • Enter key support in the cash input It also gave a 10-case test table and ran every case in a simulated browser.
Our evaluation	**Correct.** Each case gave the expected message: blank, "e", and -50 → "Please enter a valid payment amount."; 0 and 100 → "Insufficient payment. Please enter at least ₱165.00."; 165 → change ₱0.00; 165.50 → ₱0.50; 200 → ₱35.00; empty cart → "Your cart is empty. Please add at least one item." Failed attempts did not create a receipt or increase the TXN number.
Modifications we made	No code changes. Chrome does not allow typing "abc" into a number field, so the non-numeric case was tested by typing "e".

CONTEXT:
I am building a Campus Cafeteria POS web app using plain HTML, CSS, and vanilla JavaScript (index.html, style.css, script.js), with in-memory data only. Cart, totals, change computation, confirmation, receipt, and New Transaction already work for valid payments. We Use the latest file you give.

OBJECTIVE:
Add payment validation with clear on-screen error and success messages, and handle transaction-flow edge cases.

REQUIREMENTS:
1. When Confirm Payment is clicked, validate the input in this order:
  a. Cart is empty → "Your cart is empty. Please add at least one item."
  b. Blank, non-numeric (use input.validity.badInput together with the raw value), or negative → "Please enter a valid payment amount."
  c. Amount less than the total → "Insufficient payment. Please enter at least ₱X.XX." (X.XX = the current total, formatted with formatPeso)
  d. Otherwise the payment is valid: compute change and show the confirmation and receipt.
2. Exact payment (amount = total) is valid and shows change ₱0.00.
3. Decimal payments like 165.50 must work and show change ₱0.50.
4. Errors appear in a red message area near the payment input; success messages appear in green. Messages must be cleared on the next action.
5. On an invalid payment, no receipt is generated and the transaction number does not increase.
6. After a successful payment, lock the cart (disable add / + / − / Remove and Confirm Payment) until "New Transaction" is clicked, so the receipt cannot change after payment.
7. Pressing Enter in the payment input triggers Confirm Payment (keyboard support).

Figure 5. Prompt 3 as entered (part 1 of 2)

CONSTRAINTS:
- Vanilla JS only, no libraries, no localStorage.
- Put validation in its own function, validatePayment(amountInput, total), that returns { valid, message, amount }.
- Do not change working features beyond what is needed. Do not add new features beyond this list.
- Short comments; beginner-readable.

EXPECTED OUTPUT:
1. The complete updated script.js (full file), plus index.html/style.css only if they changed.
2. A brief explanation of validatePayment and the cart-lock logic.
3. A test-case table (Input | Total | Expected message/result) covering: blank, "abc"/"e", -50, 0, 100 (insufficient vs ₱165), 165 (exact), 165.50, 200, and Confirm Payment with an empty cart.

Figure 6. Prompt 3 as entered (part 2 of 2)

Prompt 4 — Review and Debug Against the 12 Required Features

Type of use: Debugging

Prompt used	The exact prompt text is shown in the screenshot below (Figure 7).
Purpose of the prompt	Check the finished app against the exam's 12 required features, the transaction flow, floating-point rounding, and console errors, and fix only real bugs without changing the structure.
AI response (summary)	The AI returned a PASS/FAIL checklist. All 12 features passed, as did the extra checks: • flow order and the cart lock after payment • rounding (e.g. 165.30 → ₱0.30 change) and message clearing • console errors (none found) It ran 25 automated checks in a simulated browser, found no bugs, and made no code changes. It noted one optional hardening: using isFinite instead of isNaN, so an unrealistic input such as 1e309 cannot show ₱Infinity.
Our evaluation	**Correct.** The result matched our own manual testing of all requirements, which also found nothing to fix.
Modifications we made	None. The optional isFinite change was not applied, because no cashier would enter such a value and the goal was to keep working behavior unchanged.

CONTEXT:
I am building a Campus Cafeteria POS web app in plain HTML, CSS, and vanilla JavaScript (index.html, style.css, script.js), in-memory data only. The app is feature-complete.

OBJECTIVE:
Review and debug my code against the exam's 12 required features and transaction flow, and fix any bugs found.

REQUIREMENTS:
Check that each of these works and report PASS/FAIL with the reason:
1. At least 5 products displayed with names and prices.
2. Add to cart through on-screen buttons.
3. Increase/decrease quantity of cart items.
4. Remove a product from the cart.
5. Subtotals and total update automatically on every cart change.
6. Clear order summary of all cart items.
7. Cash payment through a numeric input field.
8. Rejects blank, non-numeric, negative, and insufficient amounts, each with a clear message.
9. Change = Amount Paid − Total, correct for valid payments.
10. Clear payment confirmation after a successful payment.
11. Receipt with items, quantities, subtotals, total, amount paid, change, and a transaction number.
12. New Transaction clears cart, payment amount, and receipt.
Also check: flow order (Browse → Add → Review → Pay → Validate → Confirm → Receipt/New), floating-point rounding, and console errors.

CONSTRAINTS:
- Fix only actual bugs; do not redesign or add new features.
- Keep my function names and structure so I can still explain the code.
- Vanilla JS only.

EXPECTED OUTPUT:
1. A PASS/FAIL checklist for the 12 features.
2. Each bug found: cause, and the fix (before → after code).
3. The complete corrected script.js (only if changes were made).
4. A one-paragraph summary of what was changed.

- We test all the requirements and we found nothing to debug too but can you double check it too.

Figure 7. Prompt 4 as entered

Prompt 5 — README for the GitHub Repository

Type of use: Documentation

Prompt used	The exact prompt text is shown in the screenshot below (Figure 8).
Purpose of the prompt	Write a brief README.md with the project title, description, scenario, product list, features, the 7-step transaction flow, validation rules, how to run, technologies used, file structure, and an AI-Assisted Development note.
AI response (summary)	The AI returned a complete README.md under one page with every requested section, including tables for the products and the payment validation rules, and a short AI-Assisted Development note stating that AI was used for code generation and debugging and that the work was reviewed and tested.
Our evaluation	**Correct.** All required sections were present, no badges or images were used, and the validation messages match the app exactly. Note: Prompt 6 later added an images/ folder, which the README's File Structure section does not yet list.
Modifications we made	No changes were made to the README content.

CONTEXT:
I built a Campus Cafeteria POS web app, using plain HTML, CSS, and vanilla JavaScript (index.html, style.css, script.js), with in-memory data and no database. Use the latest file you give

OBJECTIVE:
Write a brief README.md for my GitHub repository.

REQUIREMENTS:
Include: project title, short description, scenario (Scenario 1 — Campus Cafeteria POS), product list with prices, features list, transaction flow (7 steps), payment validation rules, how to run (open index.html in a browser), tech used, file structure, and an "AI-Assisted Development" note saying AI was used for code generation and debugging and that the work was reviewed and tested.

CONSTRAINTS:
- Markdown, concise (under 1 page), no badges or images.

EXPECTED OUTPUT:
The complete README.md content.

Figure 8. Prompt 5 as entered

Prompt 6 — UI Polish, Product Images, and Refactor

Type of use: Refactoring / UI improvement

Prompt used	The exact prompt text is shown in the screenshot below (Figure 9).
Purpose of the prompt	Polish the interface and add product images with an emoji fallback, make the three sections clearly distinct, make the total, change, and TXN number prominent, and refactor the JavaScript (remove duplicate code, use named constants, one job per function) without changing any behavior or validation message.
AI response (summary)	The AI returned full updated index.html, style.css, and script.js with: • a PRODUCTS array (id, name, price, image, emoji); renderProducts() now builds the cards, and a missing image shows its emoji • numbered, color-coded sections (1 Menu, 2 Cart & Order Summary, 3 Payment & Receipt), CSS variables for all colors, rounded cards, soft shadows, and hover effects • a large total, a green change bar, and a TXN number badge • named constants (STORE_NAME, CURRENCY, TXN_PREFIX, MESSAGES) and shared helpers (changeQty, setMessage, invalidResult, lineHtml, showReceiptPlaceholder) • smaller functions split from renderCart, plus createTransaction, init, and setupEventListeners The main function names were kept. It also gave a list of changes, a 5-step retest checklist, and a screenshot of the new layout.
Our evaluation	**Correct.** The AI re-ran the earlier 25 checks against the refactored code. Its test caught one small difference it had introduced — the new lineHtml helper dropped the space between a receipt label and its value — and it fixed this before delivering. All checks then passed, and the layout was checked in a headless Chrome browser at laptop size.
Modifications we made	No code changes. The product photos go in the images/ folder using the required file names; any missing photo shows its emoji instead.

CONTEXT:
Campus Cafeteria POS, Plain HTML, CSS, vanilla JS (index.html, style.css, script.js), in-memory data, desktop browser. All 12 required features work. Use the latest file you give

OBJECTIVE:
Polish the UI, add product images, and refactor the code without changing behavior.

REQUIREMENTS:
1. Add an "image" field to each product. Use local files in an images/ folder: rice.jpg, fried-chicken.jpg, pork-adobo.jpg, vegetable.jpg, iced-tea.jpg, bottled-water.jpg. If an image fails to load, show a matching emoji instead (🍚 🍗 🥘 🥗 🧋 💧).
2. Product cards: image on top (same size, object-fit: cover), name, price, and an "Add to Cart" button.
3. Clean, modern look: consistent spacing, rounded cards, soft shadows, a readable font, hover effects on buttons, and green for success / red for errors. Define colors as CSS variables.
4. Make the three sections (Products, Cart/Order Summary, Payment & Receipt) clearly distinct, and make the total, change, and TXN number visually prominent.
5. Refactor JS: remove duplicate code, use named constants for repeated strings, keep one job per function, and add short comments.

CONSTRAINTS:
- No libraries, CDNs, or localStorage; must work offline.
- Do not change features, flow, or the exact validation messages.
- Keep the main function names (addToCart, renderCart, validatePayment, processPayment, renderReceipt, newTransaction).

EXPECTED OUTPUT:
1. The complete updated index.html, style.css, and script.js.
2. A short list of the UI changes and the refactor changes.
3. A 5-step retest checklist.

Show less

Figure 9. Prompt 6 as entered

Testing and Understanding of AI-Generated Code

After each prompt, the app was opened in Chrome and checked with the testing checklist before moving to the next stage. The main functions we can test and explain during checking are:

- **addToCart** — adds a product, or raises its quantity by 1 if it is already in the cart.
- **increaseQty / decreaseQty / removeFromCart** — change a cart item's quantity (never below 1) or delete the item.
- **renderCart** — redraws the cart table and recomputes every subtotal and the total amount due after any change.
- **validatePayment** — checks, in order, for an empty cart, a blank, non-numeric, or negative amount, and an amount less than the total; returns { valid, message, amount }.
- **processPayment** — shows the error for an invalid payment; for a valid one, computes change = amount paid − total, creates the transaction number, shows the confirmation and receipt, and locks the cart.
- **renderReceipt** — builds the receipt with the items, quantities, subtotals, total, amount paid, change, transaction number, and date and time.
- **newTransaction** — clears the cart, payment amount, confirmation, receipt, and messages, keeping the transaction counter.
- **roundMoney / formatPeso** — round money to 2 decimals to avoid floating-point errors and display amounts as ₱0.00.